

Machine Learning and Artificial Neural Networks: A Reference Encyclopedia

Section 43 • Introduction to Electrical Engineering — Unit Companion

CONTENTS

1	Introduction: Why Machine Learning?	2
1.1	Deterministic versus probabilistic problems	2
1.2	Supervised learning	3
1.3	A brief history	3
1.4	What neural networks are good at	3
2	Machine Learning on Embedded Hardware (TinyML)	3
2.1	Context: sub-mW computing	3
2.2	Quantitative resource estimates	4
3	The Artificial Neuron	5
3.1	The fundamental unit	5
3.2	The perceptron	5
3.3	The limitation: linear separability	6
3.4	From perceptrons to sigmoid neurons	6
3.5	Common activation functions	7
4	Network Architecture	7
4.1	Layers	7
4.2	Choosing a topology	8
4.3	Parameter counting	8
5	The Forward Solve	8
5.1	Vectorization and layer equations	8
5.2	Softmax output	9
6	Features and Feature Extraction	9
6.1	What is a feature?	9
6.2	The dimensionality problem	10
6.3	Feature extraction	10
7	Datasets	11
7.1	Generating a dataset	11
7.2	The holdout method	11
7.3	Dataset balance and the naive classifier	11

8 Training a Network	12
8.1 The cost function	12
8.2 Gradient descent and the δ -learning rule	12
8.3 Epochs and update schemes	13
8.4 The learning rate	13
9 Evaluating a Classifier	13
9.1 The confusion matrix	13
9.2 Positives and negatives (per-class view)	14
9.3 F1 exposes imbalance	15
10 Model Fit: Underfitting and Overfitting	15
10.1 Diagnosis from training curves	15
10.2 Fixes	16
11 Case Study: Digit Classification End to End	16
11.1 Setup	16
11.2 Results	16
11.3 Hidden neurons as pattern detectors	18
11.4 Scaling up, and the limits of vectorization	18
11.5 Companion training script	18
12 Deployment and Inference	19
12.1 Inference	19
12.2 Acting on the output	19
12.3 Aside: recurrent networks	19
A Backpropagation in Full	19
B Source Notes and Errata	20

1 Introduction: Why Machine Learning?

1.1 Deterministic versus probabilistic problems

Every embedded system you have built so far in this course follows the same recipe: a sensor produces a signal, a microcontroller applies a rule you wrote, and an actuator responds. If a thermometer reads above 30°C, open the water valve. This works because *temperature measurement is absolute*: the mapping from input to correct output is deterministic, and you can write it down as an `if` statement.

Now consider a voice-controlled emergency stop: the word “stop,” spoken into a microphone, must halt a machine. No two utterances of “stop” produce the same waveform — pitch, speed, accent, and background noise all vary. There is no closed-form rule mapping waveform samples to the label “stop.” The problem is *probabilistic*: we can at best say that a given waveform is very likely (or unlikely) to be the word “stop.” **Machine learning (ML)** addresses exactly this class of problem: instead of writing the rule ourselves, we provide many labeled examples and let an algorithm *tune its own internal parameters* until it reproduces the correct labels. A self-driving car is the canonical

large-scale example — camera, lidar, radar, and ultrasonic data streams are fused and classified continuously, a task no hand-written rule set can cover.

1.2 Supervised learning

In **supervised learning**, the paradigm used throughout this unit, the network is trained on a dataset for which the correct answers are known: each training example pairs a set of input *features* with a correct *class label*, and the network’s internal weights are iteratively adjusted until its predictions match the labels. (The other principal categories are *unsupervised learning*, which extracts structure from unlabeled data, and *reinforcement learning*, in which an agent learns from rewards.)

1.3 A brief history

- **1943** — McCulloch–Pitts “neuron”: first mathematical model of a neuron; started the field.
- **1962** — Rosenblatt’s *perceptron*: learned its own weight values, with a convergence proof.
- **1969** — Minsky & Papert’s book proved hard limitations of single-layer perceptron networks (see §3.3), stalling the field for a decade.
- **1982** — Hopfield networks: convergence in symmetric networks via an energy-function concept.
- **1986** — *Backpropagation of errors*: a practical method for training multilayer networks; the algorithm still used today (§8).
- **Present** — probabilistic interpretations, Bayesian and spiking networks, and the deep-learning era: as the amount of data scales, deep-network performance continues to improve.

1.4 What neural networks are good at

An artificial neural network (ANN) classifies a set of input features as one of several finite outcomes. This is most useful when the feature set is high-dimensional: image recognition (classification, localization, object detection, instance segmentation), speech and keyword recognition, vibration and motion analysis. Beyond classification, a trained network is a fast nonlinear function approximator: it can act as a *backward solver*, mapping the output of a complicated forward model back to an input — powerful for design and optimization when no analytic inverse exists.

2 Machine Learning on Embedded Hardware (TinyML)

2.1 Context: sub-mW computing

ML is often associated with data-center GPUs, but a growing class of applications runs *inference* (using an already-trained model) on microcontrollers drawing single milliwatts or less — **TinyML**. Typical on-device workloads: predictive maintenance and sensor fusion from vibration/motion signals, keyword spotting and sound recognition from audio, and object detection or face unlock from images.

Two hardware tiers matter for this course:

- **Single-board computers** (e.g., Raspberry-Pi class): faster processor, more memory, full operating system, full user interface — but 100s of mW to watts.

- **Microcontrollers** (e.g., the Arduino Nano 33 BLE Sense used in our labs): bare-metal “superloop” or RTOS firmware, limited or no UI, milliwatt-scale power. The Nano 33’s nRF52840 is built around a 32-bit ARM Cortex-M4 CPU with a floating-point unit running at 64 MHz.

Cortex-M parts are rapidly gaining ML capability: relative to a Cortex-M7, a Cortex-M55 achieves roughly $6\times$ faster inference and $7\times$ better energy efficiency, and pairing an M55 with an Ethos-U55 neural accelerator reaches roughly $50\times$ speed and $25\times$ efficiency. The workflow in all cases is: *collect data* (often on a phone or PC), *extract features and train the model* on a workstation, then *deploy the frozen model* to the microcontroller, which only ever runs forward passes.

2.2 Quantitative resource estimates

The dominant operation in a neural network is the **multiply–accumulate** (MAC): one multiplication plus one addition, $acc \leftarrow acc + w x$. A fully-connected layer taking N_{in} inputs to N_{out} outputs costs

$$\text{MACs} = N_{\text{out}} \times N_{\text{in}}, \quad \text{parameters} = \underbrace{N_{\text{out}} N_{\text{in}}}_{\text{weights}} + \underbrace{N_{\text{out}}}_{\text{biases}}. \quad (1)$$

Memory footprint is (parameters) $\times$ (bytes per parameter): 4 bytes for `float32`, 1 byte if the model is quantized to `int8`. Latency is approximately MACs/(MACs per cycle $\times f_{\text{clk}}$); a Cortex-M4 sustains on the order of one `float32` MAC per cycle in a well-written loop.

Worked Example 2.1 (Deploying the motion classifier on the Nano 33 BLE Sense)

The gesture classifier of §6 uses 33 input features, two hidden layers of 20 and 10 nodes, and 4 outputs. Using Eq. (1) layer by layer:

$$\underbrace{20 \times 33}_{660} + \underbrace{10 \times 20}_{200} + \underbrace{4 \times 10}_{40} = 900 \text{ MACs per inference,}$$

$$\text{parameters} = (660 + 20) + (200 + 10) + (40 + 4) = 934.$$

At 4 bytes each, the weights occupy $934 \times 4 \approx 3.7 \text{ kB}$ — trivially inside the nRF52840’s 256 kB RAM. At one MAC per cycle and 64 MHz, the network itself evaluates in $900/64 \text{ MHz} \approx 14 \mu\text{s}$; even with $10\times$ overhead for activation functions and memory traffic it is far faster than the 2-second sensing window, so the feature extraction (FFTs), not the network, dominates compute. Contrast a naive raw-input network with 375 inputs and one 375-node hidden layer: $375 \times 375 + 4 \times 375 \approx 1.4 \times 10^5$ MACs and $\sim 1.4 \times 10^5$ parameters ($\sim 0.57 \text{ MB}$ in `float32`) — which no longer fits comfortably. This is the quantitative case for feature extraction (§6).

3 The Artificial Neuron

3.1 The fundamental unit

An ANN is built from **neurons** (nodes) connected by **synapses** (weighted links), in loose analogy with biological neural networks — your own brain recognizes handwriting using on the order of 10^9 neurons and 10^{11} synapses. The fundamental unit computes a weighted sum of its inputs plus a **bias**, passed through an *activation function* f :

$$y = f(w_1x_1 + w_2x_2 + w_3x_3 + b) = f(\mathbf{w} \cdot \mathbf{x} + b). \quad (2)$$

The weights w_i encode how strongly each input matters; the bias b shifts the firing threshold. A large positive b gives a neuron that fires on most inputs; a large negative b gives a neuron that rarely fires.

3.2 The perceptron

The original artificial neuron, the **perceptron**, uses a hard threshold:

$$y = \begin{cases} 0, & \mathbf{w} \cdot \mathbf{x} + b \leq 0, \\ 1, & \mathbf{w} \cdot \mathbf{x} + b > 0. \end{cases} \quad (3)$$

(Equivalently, “output 1 if $\sum w_i x_i$ exceeds a threshold”; $b = -\text{threshold}$.) With binary inputs, a perceptron is a weighted vote.

Worked Example 3.1 (A decision perceptron)

Should we go to a music festival? Three binary factors: x_1 = good weather, x_2 = friends going, x_3 = we have tickets. Choose $w_1 = w_2 = 5$, $w_3 = 11$, threshold = 10 (i.e., $b = -10$). Then $x_3 = 1$ alone gives $11 > 10$ (go), while $x_1 = x_2 = 1$ without tickets gives $10 \not> 10$ (stay). The choice of weights is the decision logic: tickets alone decide. Raising the threshold to 12 changes the logic to “tickets AND at least one other factor.” A silly example, but it shows how weights and bias encode decision-making — and in a trained network, the algorithm *chooses* these numbers, not us.

Worked Example 3.2 (Perceptron logic gates)

With two binary inputs, find (w_1, w_2, b) implementing AND, OR, NAND. Verify by evaluating $\mathbf{w} \cdot \mathbf{x} + b$ at all four input pairs:

	AND: $w = (1, 1), b = -1.5$	OR: $w = (1, 1), b = -0.5$	NAND: $w = (-1, -1), b = 1.5$
(0, 0)	$-1.5 \Rightarrow 0$	$-0.5 \Rightarrow 0$	$+1.5 \Rightarrow 1$
(0, 1)	$-0.5 \Rightarrow 0$	$+0.5 \Rightarrow 1$	$+0.5 \Rightarrow 1$
(1, 0)	$-0.5 \Rightarrow 0$	$+0.5 \Rightarrow 1$	$+0.5 \Rightarrow 1$
(1, 1)	$+0.5 \Rightarrow 1$	$+1.5 \Rightarrow 1$	$-0.5 \Rightarrow 0$

Geometrically, each perceptron draws a straight *decision boundary* $w_1x_1 + w_2x_2 + b = 0$ in the input plane and outputs 1 on one side (Fig. 1). For AND the boundary is $x_1 + x_2 = 1.5$: only

(1, 1) crosses it. For OR it is $x_1 + x_2 = 0.5$: any input with at least one 1 is positive.

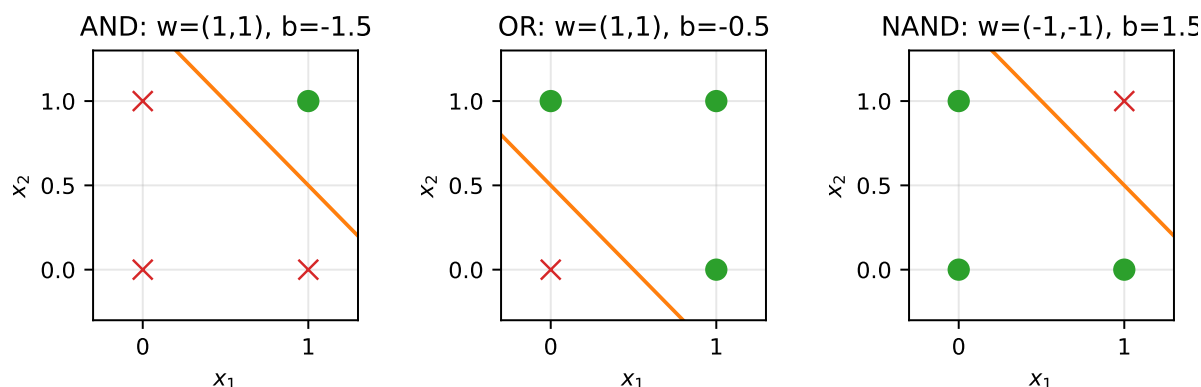


Figure 1: Perceptron decision boundaries for AND, OR, and NAND. Green circles: output 1; red crosses: output 0. The orange line is $w_1x_1 + w_2x_2 + b = 0$.

3.3 The limitation: linear separability

Since NAND is universal for digital logic, are perceptrons just more complex NAND gates? **No** — the key strength lies in the *learning algorithms* that tune the weights automatically from data. When we design a circuit we pre-solve the problem and lay out gates; with learning we specify only an architecture and let it tune itself. But a *single* perceptron can only implement functions whose 0s and 1s are separable by a straight line (a *linearly separable* function). XOR is not: no line puts (0, 1) and (1, 0) on one side and (0, 0), (1, 1) on the other (this is the Minsky–Papert limitation of 1969). The fix is *layers*: e.g., $\text{XOR}(x_1, x_2) = \text{AND}(\text{OR}(x_1, x_2), \text{NAND}(x_1, x_2))$, a two-layer network of perceptrons.

3.4 From perceptrons to sigmoid neurons

Learning requires that a *small* change in a weight produce a *small* change in the output: we then nudge each weight in the direction that reduces the error. A perceptron’s step output breaks this — an infinitesimal weight change can flip the output from 0 to 1, and that flip propagates through the network and scrambles other, previously-correct, classifications in a way that is complex to pre-calculate. You fix your misclassified “3” and break five other digits. The remedy is a smooth activation function. The **sigmoid neuron** replaces the step with

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = \mathbf{w} \cdot \mathbf{x} + b, \quad (4)$$

which behaves like the perceptron in the extremes ($\sigma(z) \rightarrow 1$ for large positive z , $\rightarrow 0$ for large negative z) but smooths out the sudden transition, and its inputs/outputs may take any value in (0, 1), not just 0 or 1. Because the output is continuous, we set a criterion to interpret it, e.g., “if the output exceeds 0.5, the neuron says yes.”

3.5 Common activation functions

Function	Definition	Range	Notes
ReLU	$f(x) = \max(0, x)$	$[0, \infty)$	cheap; sparse activation; scale-invariant; mitigates vanishing gradients for $x > 0$; “dead ReLU” if a neuron sticks at 0
Sigmoid	$\sigma(x) = 1/(1 + e^{-x})$	$(0, 1)$	probabilistic interpretation; saturates for large $ x $ (vanishing gradients)
tanh	$\tanh(x) = 2\sigma(2x) - 1$	$(-1, 1)$	zero-centered S-curve; still saturates

The ReLU (“ramp,” or — in language familiar from rectifier circuits — a half-wave rectification function) is the workhorse in this unit’s examples (Fig. 2). Its derivative is simply 1 for positive arguments and 0 otherwise, which makes training arithmetic trivial (§8).

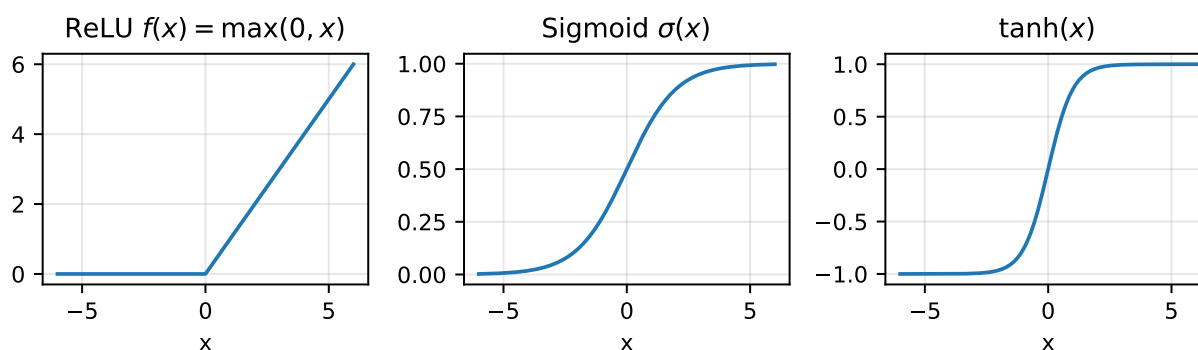


Figure 2: The three most common activation functions.

4 Network Architecture

4.1 Layers

Neurons are arranged in consecutive **layers**: an *input layer* holding the feature vector $\mathbf{x}$ (raw or normalized values), one or more *hidden layers* (their node values are visible in neither the input nor the output), and an *output layer* $\mathbf{y}$ with one node per class. In the standard *fully-connected* (multi-layer perceptron, MLP) topology, every node connects to every node of the previous layer; these are *feed-forward* networks — during operation all signals move forward, never backward. (Networks with feedback, *recurrent* neural networks, have memory and time-dependent behavior; see §12.) Training a network with at least one hidden layer is commonly called **deep learning**; the “deep” refers to the number of unit layers between input and output. As information moves through successive layers, the network is in effect making decisions about more and more abstract items — early hidden neurons might detect a pen stroke, later ones a full digit (§11).

4.2 Choosing a topology

Topology choice is driven by heuristics — “art, not science”:

- **Input width N** : fixed by the number of features (e.g., 25 for a 5×5 image, 784 for 28×28 , 4096 for 64×64).
- **Output width M** : one node per class (10 for digits 0–9). One-per-class beats a compact binary encoding: with 10 output neurons the “0” neuron just weights the stroke-detecting hidden neurons heavily, whereas 4 binary outputs would force the network to both recognize the digit *and* convert it to binary.
- **Hidden layers**: for problems at this course’s scale, no more than two hidden layers are typically needed; keep hidden widths equal and not larger than N , or training is inefficient and overfitting results (§10).

4.3 Parameter counting

With biases, the number of trainable parameters is the sum over layers of Eq. (1). For input N , hidden widths A and B , and M outputs:

$$P = (AN + A) + (BA + B) + (MB + M). \quad (5)$$

Parameters grow with the *product* of adjacent widths, so wide, deep networks rapidly accumulate parameters — and more parameters require both more training data and more computation. (Large networks train on GPUs or accelerator ASICs; the small networks here train in seconds on any laptop.)

Worked Example 4.1 (Parameter counts)

(a) The 5×5 digit network of §11: $N = 25$, one hidden layer $A = 20$, $M = 5$: $P = (20 \cdot 25 + 20) + (5 \cdot 20 + 5) = 520 + 105 = 625$. (b) MNIST-sized input, same hidden layer: $N = 784$, $A = 20$, $M = 10$: $P = (20 \cdot 784 + 20) + (10 \cdot 20 + 10) = 15,700 + 210 = 15,910$. (c) Hidden layer matched to the input, $A = 784$: $P = (784 \cdot 784 + 784) + (10 \cdot 784 + 10) = 615,440 + 7,850 = 623,290$ — a $39\times$ jump from (b), illustrating why hidden width is kept modest.

5 The Forward Solve

5.1 Vectorization and layer equations

A 5×5 image matrix is first *vectorized* — its columns stacked into a single column vector — to form the input layer $\mathbf{x}$ ($N = 25$). Each layer l then applies its weight matrix $\mathbf{W}^{(l)}$ and bias vector $\mathbf{b}^{(l)}$ and an elementwise activation:

$$\mathbf{h}^{(l)} = f_a(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}), \quad \mathbf{h}^{(0)} \equiv \mathbf{x}. \quad (6)$$

$\mathbf{W}^{(l)}$ has one row per neuron in layer l and one column per neuron in layer $l-1$ (entry $W_{a,n}^{(l)}$ connects node n of the previous layer to node a of layer l), so each layer is a matrix–vector product — a bank of MAC operations — followed by the activation. Before training, weights and biases are randomized, typically uniformly in $[-1, +1]$.

5.2 Softmax output

For the final layer $H+1$ we want a *probability distribution* over the M classes rather than raw numbers. The **softmax** function exponentiates and normalizes the raw output vector $\mathbf{y}$:

$$\sigma_m(\mathbf{y}) = \frac{e^{y_m}}{\sum_{m'=1}^M e^{y_{m'}}}. \quad (7)$$

Every element lies in $(0, 1)$ and the elements sum to unity; softmax generalizes the logistic function to M dimensions. The predicted class is the one with the highest probability. Two practical notes: (i) softmax is unchanged if a constant is subtracted from every y_m ; numerical implementations subtract $\max_m y_m$ before exponentiating so the exponentials cannot overflow; (ii) the correct answer is encoded as a *one-hot* vector $\hat{\mathbf{y}} = \mathbf{e}_d$, e.g., $[0, 1, 0, 0, 0]^\top$ for the digit 2.

Worked Example 5.1 (Forward solve by hand)

Take $N = 2$ inputs, one hidden layer of $A = 2$ ReLU nodes, $M = 2$ softmax outputs, with

$$\mathbf{W}^{(1)} = \begin{bmatrix} 1 & -1 \\ 0.5 & 2 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0.5 \\ -1 \end{bmatrix}, \quad \mathbf{W}^{(2)} = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}, \quad \mathbf{b}^{(2)} = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Hidden pre-activation: $\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} = [1+1+0.5, 0.5-2-1]^\top = [2.5, -2.5]^\top$. ReLU clips the negative entry: $\mathbf{h}^{(1)} = [2.5, 0]^\top$. Output pre-activation: $\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)} = [2.5, -2]^\top$. Softmax: with $e^{2.5} = 12.18$, $e^{-2} = 0.135$,

$$\mathbf{y} = \frac{1}{12.18 + 0.135} [12.18, 0.135]^\top = [0.989, 0.011]^\top.$$

The network predicts class 1 with 98.9% confidence. Note both probabilities are strictly positive and sum to 1.

Once trained, classification of a new input requires only this *single forward solve* through the stored weights — no further training — which is what makes deployed neural networks fast even when training was expensive (§12).

6 Features and Feature Extraction

6.1 What is a feature?

A **feature** is an individual measurable property or characteristic of the phenomenon being observed. For a grayscale image, each pixel intensity is a feature; for accelerometer motion, each sampled acceleration value could be a feature — but usually we can do much better.

6.2 The dimensionality problem

Consider classifying wrist gestures (left–right, up–down, circle, idle) from a 3-axis accelerometer sampled at 62.5 Hz over a 2-second window: $125 \text{ samples} \times 3 \text{ axes} = 375 \text{ raw values per window}$ (Fig. 3, left). Feeding all 375 raw values into a network runs into the two classic problems of deep learning: (1) computational complexity and (2) the need for lots of training data (recall Example 2.1: the raw-input network needed $\sim 150\times$ the MACs and parameters). Moreover, a single raw (x, y, z) sample is nearly useless — samples from all four gesture classes overlap in a blob around gravity.

6.3 Feature extraction

Feature extraction computes a small number of informative quantities from the raw window. The simplest useful feature is the *root-mean-square* of each axis over the window,

$$x_{\text{RMS}} = \sqrt{\frac{1}{K} \sum_{k=1}^K x_k^2}, \quad (8)$$

which measures the signal “intensity”; the 375 raw values collapse to just 3 features. Plotted in 3-D $(x_{\text{RMS}}, y_{\text{RMS}}, z_{\text{RMS}})$ space, the four gesture classes form well-separated clusters — the model’s job becomes easy.

Motion also lives in the *frequency domain*: an FFT of each axis reveals the periodicity of the gesture, and the *power spectral density* (PSD) shows how signal power is distributed over frequency (Fig. 3, right). The lab classifier uses 11 features per axis — RMS, the 3 largest PSD peak amplitudes, their 3 frequencies, and 4 spectral power bins (0.1–0.5, 0.5–1, 1–2, 2–5 Hz) — i.e., 33 features total: a $375 \rightarrow 33$ reduction that *keeps* the discriminative information and discards redundancy.

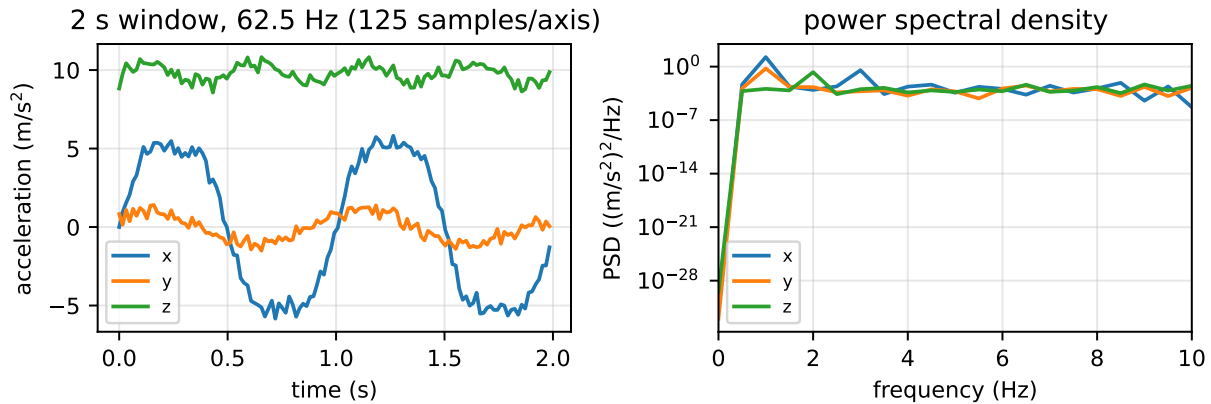


Figure 3: A synthetic 2-s, 62.5-Hz accelerometer window (left) and its PSD (right). The x -axis oscillation at ~ 1 Hz dominates the PSD; the z axis sits at gravity with a small 2-Hz component. RMS values for this window: $x_{\text{RMS}} = 4.26$, $y_{\text{RMS}} = 0.81$, $z_{\text{RMS}} = 9.81 \text{ m/s}^2$.

Worked Example 6.1 (RMS as a feature)

A 1-second, 4-sample-per-second window of x -axis acceleration reads $x = [-2, 3, -1, 2] \text{ m/s}^2$. The mean is 0.5 m/s^2 — nearly blind to the motion, since oscillation averages out. The RMS,

$$x_{\text{RMS}} = \sqrt{\frac{1}{4}(4 + 9 + 1 + 4)} = \sqrt{4.5} = 2.12 \text{ m/s}^2,$$

captures the motion intensity. An idle window $[0.1, -0.1, 0.1, -0.1]$ gives $x_{\text{RMS}} = 0.1$: one number already separates “moving” from “idle.” In Python:

```
import numpy as np
x = np.array([-2.0, 3.0, -1.0, 2.0])
rms = np.sqrt(np.mean(x**2)) # 2.1213
```

7 Datasets

7.1 Generating a dataset

A dataset pairs many feature sets with one class label each (at least two examples, ideally many more). For the digit task, the MNIST benchmark provides 60,000 training images of 28×28 grayscale handwritten digits (scanned from over 200 people — half government employees, half high-school students) plus 10,000 test images *collected from different people*, so the test measures true generalization. If the number of unique classes is large, care must be taken to sample the system properly when generating training data; otherwise the network will generally perform poorly.

7.2 The holdout method

To tune and honestly evaluate a model, randomly divide the dataset:

Split	Typical share	Used for
Training set	60%	updating the model parameters
Validation set	20%	checking the model on unseen data while <i>tweaking hyperparameters</i> and repeating training
Test set	20%	one final evaluation of the finished model

Parameters (weights, biases) are learned by training; **hyperparameters** (learning rate, layer sizes, feature choices, preprocessing settings) are set by the designer and tuned against the validation set. The test set is touched once — if you tune against it, it silently becomes a second validation set and your reported accuracy is optimistic.

7.3 Dataset balance and the naive classifier

Accuracy alone can lie. Train an image classifier on 1000 photos of which 98.9% are “field” and 1.1% are “dog”: a model that answers “*field*” for every input — a **naive classifier** — scores 99% accuracy while learning nothing. Balanced classes (or metrics that expose imbalance, §9) are essential.

8 Training a Network

8.1 The cost function

Training needs a scalar objective to optimize. The obvious candidate, $C(\mathbf{w}, \mathbf{b}) = \text{number of correctly classified images}$, fails: it is not a smooth function of the weights and biases — tiny weight changes usually change it not at all, then suddenly by a whole count — so there is no gradient to follow. Instead we use a smooth **cost (loss) function** of the continuous outputs. In this unit we track, per epoch, the accumulated L1 error

$$C_{\text{total}} = \sum_{j=1}^J \sum_{m=1}^M |\hat{y}_m^{(j)} - y_m^{(j)}|, \quad (9)$$

where $\hat{\mathbf{y}}$ is the one-hot correct label, $\mathbf{y}$ the softmax output, and J the number of training instances. C_{total} is a convenient scalar measure of progress from epoch to epoch; one may define a cost threshold below which the network is considered trained.

8.2 Gradient descent and the δ -learning rule

Training iterates two steps per instance: (a) the *forward solve* (§5), then (b) *backpropagation*, which nudges every weight and bias downhill on the cost surface with step size set by the **learning rate** α . First the output error vector is formed,

$$\boldsymbol{\varepsilon} = \hat{\mathbf{y}} - \mathbf{y}, \quad (10)$$

and used directly as the output layer’s “delta,” $\boldsymbol{\delta}^{(H+1)} = \boldsymbol{\varepsilon}$. Each layer’s weights and biases are then adjusted in proportion to its delta and its input:

$$\Delta \mathbf{W}^{(l)} = \alpha \boldsymbol{\delta}^{(l)} [\mathbf{h}^{(l-1)}]^\top, \quad \Delta \mathbf{b}^{(l)} = \alpha \boldsymbol{\delta}^{(l)}, \quad \mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} + \Delta \mathbf{W}^{(l)}, \quad (11)$$

and each hidden layer receives its share of the blame by passing the next layer’s delta *backward through the same weights* (traversed in reverse) and gating it by whether each ReLU was active. That is the essence of **backpropagation**: errors flow backward along the very links the signals flowed forward along. The precise hidden-layer formula (with the transposed weight matrix and the elementwise — Hadamard — product) is derived in Appendix A; for ReLU the activation derivative is simply 1 for positive arguments and 0 otherwise, so the gating is a mask. Adjustments may be positive or negative depending on the sign of the error.

Worked Example 8.1 (One backpropagation step at the output layer)

Continue Example 5.1: the forward solve gave $\mathbf{h}^{(1)} = [2.5, 0]^\top$ and $\mathbf{y} = [0.989, 0.011]^\top$. Suppose the correct label is class 2, $\hat{\mathbf{y}} = [0, 1]^\top$, and $\alpha = 0.1$. Then

$$\begin{aligned} \boldsymbol{\delta}^{(2)} = \boldsymbol{\varepsilon} &= [0 - 0.989, 1 - 0.011]^\top = [-0.989, +0.989]^\top, \\ \Delta \mathbf{W}^{(2)} &= 0.1 \begin{bmatrix} -0.989 \\ +0.989 \end{bmatrix} \begin{bmatrix} 2.5 & 0 \end{bmatrix} = \begin{bmatrix} -0.247 & 0 \\ +0.247 & 0 \end{bmatrix}, \quad \Delta \mathbf{b}^{(2)} = \begin{bmatrix} -0.0989 \\ +0.0989 \end{bmatrix}. \end{aligned}$$

Interpretation: the weight and bias feeding output node 1 (wrongly confident) are pushed *down*; those feeding node 2 are pushed *up*; and the second column of $\Delta \mathbf{W}^{(2)}$ is zero because hidden

node 2 was inactive ($h_2^{(1)} = 0$) — a link whose input carried no signal takes no blame. Note the bias still updates: it is a weight on a constant input of 1.

8.3 Epochs and update schemes

An **epoch** updates the weights using every instance in the training set once. Three standard schemes:

1. **Sequential / stochastic gradient descent (SGD)**: backpropagate after every single instance. Simple; used in this unit's examples.
2. **Batch**: compute errors for the entire training set, then backpropagate once per epoch using the average error. Cheaper per epoch, but convergence takes longer at the same α .
3. **Mini-batch**: update after each subset of the training set — the de facto standard at scale, balancing gradient quality against the parallel efficiency of modern hardware.

8.4 The learning rate

α is the most important hyperparameter. Too small and convergence is needlessly slow; too large and updates overshoot, causing unstable learning or convergence to a poor local minimum of the cost landscape. In the digit case study (§11), the cost after 10^3 epochs spans roughly two orders of magnitude between $\alpha = 0.001$ and $\alpha = 0.05$ — but pushing to $\alpha = 0.5$ degrades training (Fig. 6, right).

9 Evaluating a Classifier

9.1 The confusion matrix

Run the trained model over the validation set and tally results in a **confusion matrix**: rows are *actual* labels, columns are *predicted* labels; each validation sample increments one cell. Correct predictions land on the diagonal. For the 1000-sample gesture validation set:

Actual	Predicted			
	Circle	Idle	Left–Right	Up–Down
Circle	205	10	1	46
Idle	6	199	0	32
Left–Right	9	17	223	34
Up–Down	21	8	3	186

Total accuracy is the diagonal sum over the grand total: $(205 + 199 + 223 + 186)/1000 = 813/1000 = 0.813$. Off-diagonal structure is diagnostic: here many gestures are misread as Up–Down (the 46/32/34 column), pointing to a specific weakness rather than uniform error.

9.2 Positives and negatives (per-class view)

Pick one class as “positive” (everything else is “negative”). Each prediction is then one of:

- **TP** (true positive): predicted positive, actually positive;
- **TN** (true negative): predicted negative, actually negative;
- **FP** (false positive, Type I error): predicted positive, actually negative;
- **FN** (false negative, Type II error): predicted negative, actually positive.

From these, four standard metrics:

$$ACC = \frac{TP + TN}{TP + TN + FP + FN} \quad \text{per-class accuracy} \quad (12)$$

$$TPR = \frac{TP}{TP + FN} \quad \text{sensitivity, recall, hit rate} \quad (13)$$

$$TNR = \frac{TN}{TN + FP} \quad \text{specificity, selectivity} \quad (14)$$

$$PPV = \frac{TP}{TP + FP} \quad \text{precision} \quad (15)$$

and their harmonic-mean summary, the **F1 score**:

$$F_1 = 2 \frac{PPV \cdot TPR}{PPV + TPR} = \frac{2TP}{2TP + FP + FN}. \quad (16)$$

Worked Example 9.1 (Metrics for the Idle class)

Treat Idle as positive in the matrix above. $TP = 199$ (Idle row, Idle column). $FN = 6 + 0 + 32 = 38$ (Idle row, other columns). $FP = 10 + 17 + 8 = 35$ (Idle column, other rows). $TN = 1000 - 199 - 38 - 35 = 728$ (everything else). Then

$$\begin{aligned} ACC &= \frac{199 + 728}{1000} = 0.927, & TPR &= \frac{199}{199 + 38} = 0.840, \\ TNR &= \frac{728}{728 + 35} = 0.954, & PPV &= \frac{199}{199 + 35} = 0.850, \\ F_1 &= 2 \cdot \frac{0.850 \times 0.840}{0.850 + 0.840} = 0.845. \end{aligned}$$

Repeating for all classes (Circle, Idle, Left–Right, Up–Down) gives:

$$\text{per-class } ACC = (0.907, 0.927, 0.936, 0.856), \quad F_1 = (0.815, 0.845, 0.875, 0.721),$$

with total accuracy 0.813 and F1 average 0.818. Up–Down’s low F1 (0.721) flags it as the weak class — consistent with the column structure noted above.

9.3 F1 exposes imbalance

Now the naive-classifier trap in numbers. Suppose a lopsided validation set (973 Idle, 27 gestures) and a model that predicts Idle for everything:

	Circle	Idle	Left-Right	Up-Down
Circle	0	6	0	0
Idle	0	973	0	0
Left-Right	0	11	0	0
Up-Down	0	10	0	0

Total accuracy is $973/1000 = 0.973$ — superb, apparently. Per-class accuracies are all high too, $(0.994, 0.973, 0.989, 0.99)$, because TN dominates Eq. (12). But every non-Idle class has $TP = 0$, hence $F_1 = 0$, and the **F1 average is 0.247**. A high accuracy with a collapsed F1 average is the signature of an imbalanced dataset and/or a naive model: report both.

10 Model Fit: Underfitting and Overfitting

10.1 Diagnosis from training curves

Plot accuracy (or loss) versus epoch for the training set *and* the validation set (Fig. 4):

- **Underfit**: both curves poor — the model fails to capture the trends in the data (decision boundary too simple).
- **Good fit**: both curves good and close together — the model captures trends and generalizes to unseen data.
- **Overfit**: training accuracy high but validation accuracy stalls or degrades (equivalently, validation loss turns back upward) — the model, with many parameters and a small training set, has *memorized* the training data rather than generalizing from it. The decision boundary wiggles around individual training points.

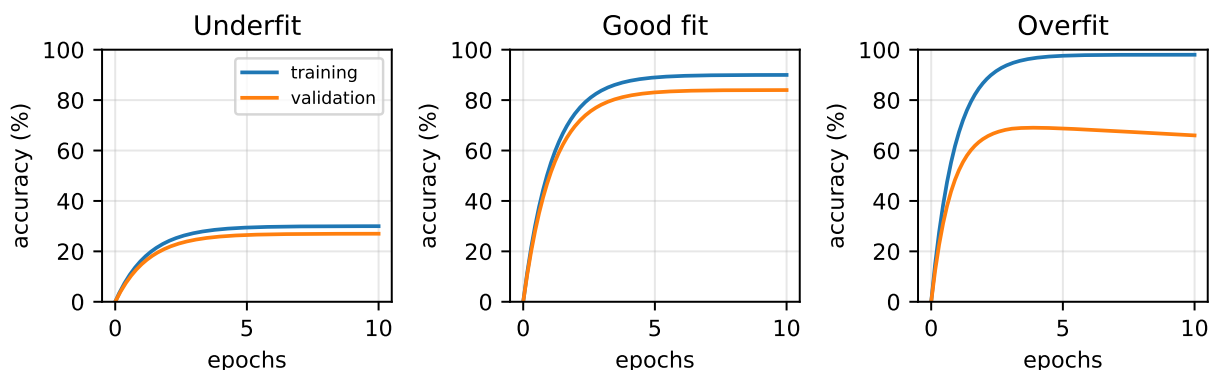


Figure 4: Accuracy-versus-epoch signatures of underfitting, good fit, and overfitting. In the overfit panel the gap between training and validation grows with training time; early stopping would halt near the validation peak.

10.2 Fixes

Fixing underfit: get more data; try different or more features; train longer; try a more complex model (more layers, more nodes).

Fixing overfit (countermeasures collectively termed **regularization**):

- get more data;
- **early stopping:** halt training when validation performance stops improving;
- reduce model complexity to the smallest network that fits the problem;
- add an L1 or L2 weight-norm penalty term to the cost function (discourages large weights);
- **dropout** (neural networks): randomly deactivate a fraction (typically 20–50%) of hidden nodes during each training step, so the network cannot rely on any single pathway. Dropout is applied only during training; at inference time all nodes are active.

11 Case Study: Digit Classification End to End

11.1 Setup

We train a network to identify the digits 1–5 from tiny monochrome images: each digit is drawn on a 5×5 grid of pixels valued 0 (background) or 1 (stroke); the five images in Fig. 5 constitute the entire training set. This toy problem contains, in miniature, every ingredient of practical image classification.

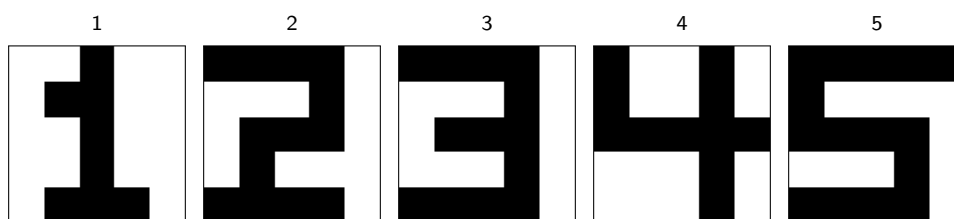


Figure 5: The training dataset: digits 1–5 as 5×5 binary images.

Following §§4–5: vectorize each image ($N = 25$), one hidden layer of $A = 20$ ReLU nodes (fewer than 25), $M = 5$ softmax outputs, one-hot labels $\hat{\mathbf{y}} = \mathbf{e}_d$; 625 trainable parameters (Eq. 5). Training is SGD exactly as in §8: each epoch presents all five images in sequence, one forward solve and one backpropagation update per image, for 10^3 epochs at each of four learning rates $\alpha \in \{0.001, 0.005, 0.01, 0.05\}$, tracking C_{total} (Eq. 9) every epoch.

11.2 Results

Figure 6 (left) shows the learning curves. As predicted, larger learning rates converge faster: the final cost spans roughly two orders of magnitude between $\alpha = 0.001$ ($C \approx 0.62$) and $\alpha = 0.05$ ($C \approx 0.0065$). This does *not* mean α should grow without bound — rerunning with $\alpha = 0.5$ (Fig. 6, right) shows the noisy, stalling behavior of overshoot.

After training, each learned weight set is saved to disk; classifying a new image then needs only a single forward solve. To test *generalization*, we construct a corrupted “4” — one stroke pixel deleted,

one spurious background pixel added — so the test image matches no training image exactly (Fig. 7, left). All four trained networks classify it correctly, e.g.:

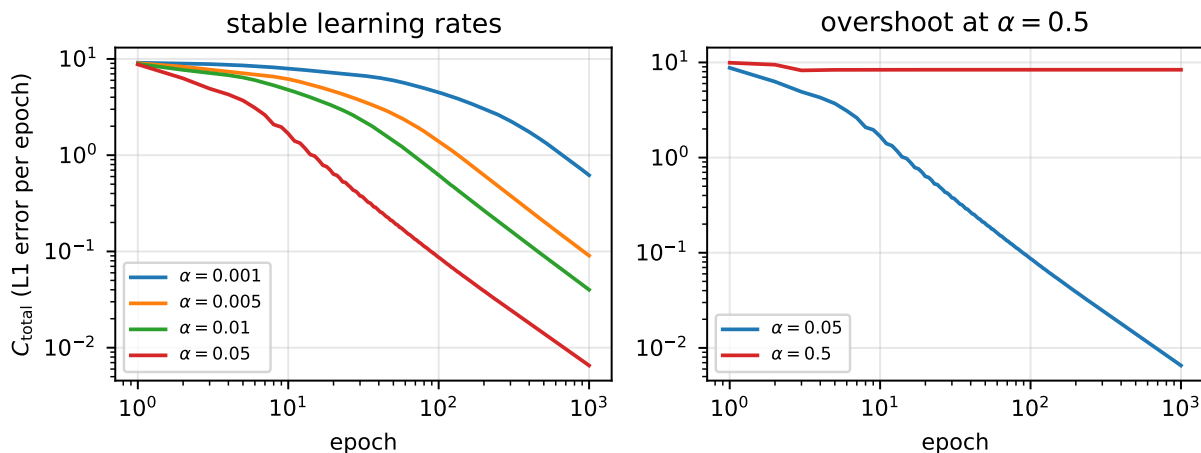


Figure 6: Left: total L1 error per epoch (log-log) for four stable learning rates. Right: $\alpha = 0.5$ overshoots and stalls. Regenerated from the companion script `ann_digits.py` (Listing 1).

α	prediction	softmax confidence
0.001	“4”	0.79
0.005	“4”	0.95
0.01	“4”	0.97
0.05	“4”	0.99

a first demonstration of the noise robustness that makes neural networks attractive for pattern recognition.

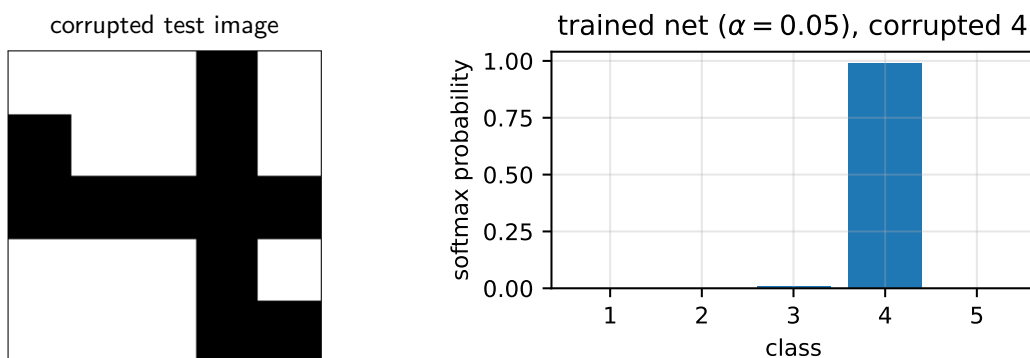


Figure 7: Left: the corrupted test “4.” Right: softmax output of the $\alpha = 0.05$ network for this image — class 4 wins with probability 0.99.

11.3 Hidden neurons as pattern detectors

Why does this work? A useful heuristic: each hidden neuron detects whether a certain sub-pattern is present. To make one neuron fire on a particular stroke, weight the pixels making up that stroke heavily and reduce (or make negative) the weights of the other pixels — exactly the four-pixel example of Eq. (2) scaled up. If four hidden neurons detect the four arcs of a “0,” the “0” output neuron simply weights those four hidden outputs strongly. (A toy picture — don’t get carried away poking holes in it — but it explains why one-output-per-class beats binary encoding: the class neuron reads pattern detectors directly instead of also computing a binary conversion.)

11.4 Scaling up, and the limits of vectorization

Real image classification differs mainly in size, not in kind: MNIST’s 28×28 images give 784 input nodes, 10 classes, 60,000 training images, and the same vectorize–forward–solve–backpropagate machinery applies unchanged. But vectorizing discards the image’s two-dimensional structure — neighboring pixels become distant vector elements — and the fully-connected parameter count explodes (623,290 parameters for a 784-node hidden layer). Both problems motivate *convolutional* neural networks, which restrict select weights to zero in a structured way (structured sparsity) and share weights across image positions; they are the standard tool beyond this unit’s scope.

11.5 Companion training script

Listing 1 is a condensed Python implementation (the full companion script `ann_digits.py` also generates every figure in this document).

Listing 1: Core of the digit-classification trainer (SGD, ReLU + softmax, delta rule, with biases).

```
import numpy as np
def relu(z): return np.maximum(0.0, z)
def drelu(z): return (z > 0).astype(float)
def softmax(y):
    e = np.exp(y - y.max()) # subtract max: overflow guard
    return e / e.sum()

def train(X, Y, alpha, epochs=1000, A=20, seed=1):
    r = np.random.default_rng(seed)
    N, M = X[0].size, Y[0].size
    W1 = r.uniform(-1, 1, (A, N)); b1 = r.uniform(-1, 1, A)
    W2 = r.uniform(-1, 1, (M, A)); b2 = r.uniform(-1, 1, M)
    cost = np.zeros(epochs)
    for ep in range(epochs):
        for x, yhat in zip(X, Y): # one epoch = all instances
            z1 = W1 @ x + b1; h = relu(z1) # forward solve
            y = softmax(W2 @ h + b2)
            eps = yhat - y # output error
            cost[ep] += np.abs(eps).sum() # L1 cost metric
            d2 = eps # delta rule, output layer
            d1 = (W2.T @ d2) * drelu(z1) # backpropagate to hidden
            W2 += alpha * np.outer(d2, h); b2 += alpha * d2
            W1 += alpha * np.outer(d1, x); b1 += alpha * d1
    return (W1, b1, W2, b2), cost
```

12 Deployment and Inference

12.1 Inference

Inference is using the trained model to make predictions on unseen data in the wild. The deployed pipeline mirrors training minus backpropagation: collect data $\rightarrow$ preprocess $\rightarrow$ extract features $\rightarrow$ one forward solve $\rightarrow$ class probabilities. On the Nano 33 BLE Sense gesture system, a 2-s window sampled at 62.5 Hz (375 raw values) is reduced to 33 features and fed to the model, which returns, e.g.,

$$P(\text{left-right}) = 0.9143, \quad P(\text{up-down}) = 0.0032, \quad P(\text{circle}) = 0.0581, \quad P(\text{idle}) = 0.0244.$$

12.2 Acting on the output

The classifier's probabilities gate ordinary embedded code:

```
if (p_left_right > 0.5) { /* do stuff */ }
if (p_up_down > 0.5) { /* do some things */ }
if (p_circle > 0.8) { /* and now for something different */ }
```

The thresholds are engineering choices: raising a threshold (0.8 for circle) trades sensitivity (TPR) for precision (PPV) — exactly the metrics of §9. Actions with costly false positives get high thresholds.

12.3 Aside: recurrent networks

Feed-forward networks evaluate each window independently. In a **recurrent neural network** (RNN) the output of a neuron is also an input — unfolded in time, the hidden state h_t feeds h_{t+1} — giving the network *memory* and time-dependent behavior (necessary because a neuron's output can't feed back into an instantaneous evaluation). RNNs suit sequential data (speech, text) and are beyond this unit's scope.

A Backpropagation in Full

This appendix fills in the hidden-layer update quoted in §8, for students comfortable with Section 42's linear algebra. With pre-activations $\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}$, the delta vectors are computed from the output backward:

$$\boldsymbol{\delta}^{(H+1)} = \hat{\mathbf{y}} - \mathbf{y}, \quad \boldsymbol{\delta}^{(l)} = \left\{ [\mathbf{W}^{(l+1)}]^\top \boldsymbol{\delta}^{(l+1)} \right\} \odot f'_a(\mathbf{z}^{(l)}), \quad (17)$$

where $\odot$ is the *Hadamard* (elementwise) product and $[\mathbf{W}^{(l+1)}]^\top$ redistributes the next layer's blame backward across the same links, traversed in reverse. For ReLU, $f'_a(z) = 1$ for $z > 0$ and 0 otherwise (the derivative of $\max(0, z)$ is undefined exactly at $z = 0$; by convention the value 0 is assigned there — in code, the mask $(\mathbf{z} > 0)$). Each layer then updates by the outer product of its delta with its input vector, $\Delta \mathbf{W}^{(l)} = \alpha \boldsymbol{\delta}^{(l)} [\mathbf{h}^{(l-1)}]^\top$ and $\Delta \mathbf{b}^{(l)} = \alpha \boldsymbol{\delta}^{(l)}$, producing a matrix of the same dimensions as the weight matrix.

Two remarks. (i) Verify Eq. (17) on Example 8.1: $\boldsymbol{\delta}^{(1)} = \{ [\mathbf{W}^{(2)}]^\top \boldsymbol{\delta}^{(2)} \} \odot f'_a(\mathbf{z}^{(1)})$ with $[\mathbf{W}^{(2)}]^\top \boldsymbol{\delta}^{(2)} = [-0.989 - 0.989, 0.989]^\top = [-1.978, 0.989]^\top$ and $f'_a(\mathbf{z}^{(1)}) = [1, 0]^\top$ (since $\mathbf{z}^{(1)} = [2.5, -2.5]^\top$), so

$\delta^{(1)} = [-1.978, 0]^\top$: the dead ReLU neuron receives no update. (ii) Although Eq. (9) tracks an L1 error as the training *metric*, the update rule itself is exactly gradient descent on the cross-entropy loss $L = -\sum_m \hat{y}_m \ln y_m$: for a softmax output layer, $\partial L / \partial z_m = y_m - \hat{y}_m$, so setting $\delta^{(H+1)} = \hat{\mathbf{y}} - \mathbf{y}$ and applying the updates above descends the cross-entropy gradient exactly — the δ -learning rule as used here is rigorous, not merely heuristic.

B Source Notes and Errata

Differences from, and corrections to, the source material:

- **Bias terms added throughout.** The section notes formulate the network bias-free, $\mathbf{h}^{(l)} = f_a(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)})$, while the lecture slides include the bias b in the fundamental neuron equation. This document uses the biased form (Eq. 6) consistently; all worked examples, parameter counts, and the companion script include bias vectors, so parameter counts here exceed the notes' weight-only counts (e.g., 625 vs. 600 for the digit network).
- Several figure cross-references in the notes render as “Fig. ” with a missing number (broken \ref); the corresponding figures are regenerated and numbered here.
- The final lecture slide of Lectures 19–20 (“How to train your artificial neural network”) repeats the same bullet block twice — the second block, introducing an alternative smooth cost function, was evidently intended to differ; §8 supplies the intended content.
- Lecture 17–18 title slide: “Why use NN when you can hardcode” should read “...when *you* can hardcode.”
- The notes' `Softmax()` helper omits the subtract-max overflow guard for clarity; Listing 1 includes it.